

HomeWork

DevOps

HW_L4_01_Git

۱. فایل تمرین را در پنل خود آپلود کنید.

۲. فایل تمرین به صورت (نام تمرین+نام و نام خانوادگی) به انگلیسی باشد. title

نمونه: AmirMohammadMohammadi_git_01

۳. در صورتی که سوال و یا ابهامی دارید در گروه چت تلگرامی بپرسید.

تمرین Git

مجموع امتیاز: سی نمره

- تمام تمرین‌ها روی سیستم لوکال قابل اجرا هستند
- تحویل یک پوشه کامل شامل repository ها، فایل‌ها و خروجی دستورات
- تمام دستورات git را در فایل git_commands_log.txtذخیره کنید

پیش‌نیاز Git **:**باید نصب شده باشد

ساختار پیشنهادی:

```
git_homework/
├── 01_basics/
├── 02_commits/
├── 03_branching/
├── 04_merging/
├── 05_stash/
├── 06_diff_history/
├── 07_reset_revert/
├── 08_github/
└── git_commands_log.txt
```

بخش اول: اصول اولیه — Git چهار نمره

سوال ۱.۱ — نصب و معرفی — یک نمره

آنچه می‌خواهم :نصب Git و معرفی خود بهGit

فایل 01_basics/setup.mdبا توضیح:

- نحوه نصب Git در سیستم عامل شما

دانشکار

- دستورات معرفی نام و ایمیل بهGit
- بررسی تنظیمات با git config --list

دستورات:

#نصب) Git متناسب با سیستم عامل(

Ubuntu/Debian: sudo apt install git

CentOS/RHEL: sudo yum install git

macOS: brew install git

#معرفی خود

git config --global user.name "Your Name"

git config --global user.email "your.email@example.com"

#بررسی تنظیمات

git config --list

git config user.name

git config user.email

تحویل :خروجی git config --listدر فایل 01_basics/config_output.txt

دانشکار

سوال ۱.۲ — ایجاد Repository و اولین — Commit سه نمره

آنچه می‌خواهم :ایجاد repository و درک مفهومatomic commit

فایل 01_basics/concepts.mdبا توضیح:

- **Repository:**مخزن Git که تاریخچه تغییرات را نگه می‌دارد
- **Atomic Commit:**کامیت اتمی یعنی هر کامیت یک تغییر منطقی و کامل را نشان می‌دهد
- **Nested Repository:**چرا نباید repository داخل repository ایجاد کرد

سناریو:

```
cd 01_basics

mkdir my_project && cd my_project

git init



#ایجاد فایل‌ها

echo "# My Project" > README.md

echo "console.log('Hello World');" > app.js

echo "body { color: blue; }" > style.css
```


دانشکار

Stage کردن فایل‌ها

```
git add README.md

git status
```

Commit

```
git commit -m "Add README file"
```

Stage و commit بقیه فایل‌ها

```
git add app.js style.css

git commit -m "Add application files"
```

#نمایش تاریخچه

```
git log --oneline
```

تحویل:

- خروجی git status بعد از هر git add
- خروجی git log --oneline
- توضیح در concepts.md درباره atomic commit

دانشکار

بخش دوم: کار با Commits — چهار نمره

سوال ۲.۱ git add — و git commit — دو نمره

آنچه می‌خواهم :درک تفاوت بین working directory ، staging areaو repository

سناریو:

```
cd 02_commits

mkdir commit_demo && cd commit_demo

git init
```

#ایجاد چند فایل

```
echo "Version 1" > file1.txt

echo "Version 1" > file2.txt

echo "Version 1" > file3.txt
```

Stage #کردن یک فایل

```
git add file1.txt

git status
```


دانشکار

Stage #کردن بقیه

```
git add file2.txt file3.txt

git commit -m "Initial commit with three files"
```

#تغییر فایل‌ها

```
echo "Version 2" >> file1.txt

echo "Version 2" >> file2.txt
```

Stage #فقط file1

```
git add file1.txt

git status
```

Commit #فقط file1

```
git commit -m "Update file1 to version 2"
```

#نمایش تفاوت

git diff

git diff --staged

تحویل:

- خروجی git statusدر مراحل مختلف
- خروجی git diffو git diff --staged
- توضیح تفاوت بین working directory وstaging area

سوال ۲.۲ — اصلاح آخرین — Commit دو نمره

آنچه می‌خواهم :استفاده از git commit --amend

سناریو:

cd 02_commits

mkdir amend_demo && cd amend_demo

git init

Commit #اول

echo "Initial content" > document.txt

git add document.txt

```
git commit -m "Add document"
```

```
#فراموش کردیم یک فایل اضافه کنیم

echo "Config data" > config.txt

git add config.txt

git commit --amend -m "Add document and config file"
```

```
#یا تغییر پیامcommit

git commit --amend -m "Initial commit: Add document and config"
```

```
#نمایش تاریخچه

git log --oneline
```

تحویل:

- خروجی git logقبل و بعد ازamend
- توضیح در amend_explanation.mdدرباره زمان استفاده ازamend

بخش سوم — Branching :پنج نمره

دانشکار

سوال ۳.۱ — ایجاد و تغییر Branch — دو نمره

آنچه می‌خواهم :درک مفهومbranch ، master/mainوHEAD

فایل 03_branching/branching_concepts.mdبا توضیح:

- **Branch:**شاخه‌ای از تاریخچه که می‌تواند مستقل توسعه یابد
- **Master/Main:**شاخه پیش‌فرض درrepository
- **HEAD:**اشاره‌گر به آخرین commit در branch فعلی

سناریو:

```
cd 03_branching

mkdir branch_demo && cd branch_demo

git init


#ایجاد فایل وcommit

echo "Main content" > main.txt

git add main.txt

git commit -m "Initial commit on main"
```


دانشکار

#ایجاد branch جدید

```
git branch feature-login

git branch
```

#تغییر به branch جدید

```
git checkout feature-login
```

#یا با switch (Git 2.23+)

```
git switch feature-login
```

#ایجاد تغییرات در branch جدید

```
echo "Login feature" > login.js

git add login.js

git commit -m "Add login feature"
```

#بازگشت به main

```
git switch main
```

#نمایش تفاوت branches

```
git log --oneline --graph --all
```

تحویل:

- خروجی git branch وgit log --graph --all
- توضیح مفهوم HEAD در branching_concepts.md

سوال ۳.۲ — مدیریت Branch — سه نمره

آنچه می‌خواهم :تغییر نام، حذف و جابه‌جایی بین branches

سناریو:

```
cd 03_branching

mkdir branch_management && cd branch_management

git init
```

#ایجاد چند branch

```
echo "Content" > file.txt

git add file.txt

git commit -m "Initial commit"
```


دانشکار

```
git branch feature-a

git branch feature-b

git branch old-feature


#تغییر نام branch

git branch -m old-feature new-feature


#حذف branch (باید در branch دیگری باشیم)

git switch main

# git branch -d feature-a حذف اگر merge شده

# git branch -D feature-b حذف اجباری


#جابه‌جایی با فایل‌های uncommitted

git switch new-feature

echo "New content" > new.txt

git add new.txt
```


دانشکار

#تغییر branch (Git به صورت خودکار stash می‌کند یا خطا می‌دهد)

```
git switch main

git status
```

#بازگشت وcommit

```
git switch new-feature

git commit -m "Add new file"
```

تحویل:

- خروجی git branchدر مراحل مختلف
- توضیح تفاوت d-و D-در branch_management.md

بخش چهارم — Merging :پنج نمره

سوال ۴.۱ — Fast Forward Merge — یک نمره

آنچه می‌خواهم :درکfast forward merge

سناریو:

دانشکار

```
cd 04_merging

mkdir ff_merge && cd ff_merge

git init


# Commit در main

echo "Main file" > main.txt

git add main.txt

git commit -m "Initial commit"


#ایجاد branch وcommit

git branch feature

git switch feature

echo "Feature file" > feature.txt

git add feature.txt

git commit -m "Add feature"


#بازگشت به main وmerge
```

```
git switch main

git merge feature
```

```
#نمایش تاریخچه

git log --oneline --graph
```

تحویل:

- خروجی git log --graph بعد ازmerge
- توضیح fast forward merge در merge_types.md

سوال ۴.۲ — بدون — Conflict دو نمره

آنچه می‌خواهم merge **:**کردن branches با تغییرات مختلف

سناریو:

```
cd 04_merging

mkdir no_conflict && cd no_conflict

git init


# Setup

echo "Base" > base.txt
```


دانشکار

```
git add base.txt

git commit -m "Base commit"


# Branch A

git branch branch-a

git switch branch-a

echo "File A" > file_a.txt

git add file_a.txt

git commit -m "Add file A"


# Branch B

git switch main

git branch branch-b

git switch branch-b

echo "File B" > file_b.txt

git add file_b.txt

git commit -m "Add file B"
```

```
# Merge

git switch main

git merge branch-a

git merge branch-b


git log --oneline --graph --all
```

تحویل:

- خروجی git log --graph --all
- توضیح merge commit در merge_types.md

سوال ۴.۳ — حل Conflict — دو نمره

آنچه می‌خواهم :حل conflict با terminal وVSCode

سناریو:

```
cd 04_merging

mkdir conflict_demo && cd conflict_demo

git init


# Setup
```

```
echo "Line 1" > conflict.txt

git add conflict.txt

git commit -m "Initial commit"


# Branch A

git branch branch-a

git switch branch-a

echo "Line 2 from A" >> conflict.txt

git add conflict.txt

git commit -m "Add line from A"


# Branch B

git switch main

git branch branch-b

git switch branch-b

echo "Line 2 from B" >> conflict.txt

git add conflict.txt

git commit -m "Add line from B"
```

Merge و ایجاد conflict

```
git switch main

git merge branch-a

git merge branch-b  # Conflict!
```

#حل conflict (terminal)

```
cat conflict.txt
```

#ویرایش فایل و حل conflict

```
git add conflict.txt

git commit -m "Merge branch-b, resolve conflict"
```

#یا استفاده از VSCode

#باز کردن VSCode و استفاده از merge editor

تحویل:

- محتوای فایل conflict.txtقبل و بعد از حلconflict
- اسکرین‌شات از) VSCode merge editor اختیاری(

دانشکار

- توضیح مراحل حل conflict در conflict_resolution.md

بخش پنجم — Stash :سه نمره

سوال ۵.۱ Stash — و — Apply سه نمره

آنچه می‌خواهم :استفاده از stash برای ذخیره موقت تغییرات

سناریو:

```
cd 05_stash

mkdir stash_demo && cd stash_demo

git init



# Setup

echo "Initial" > file.txt

git add file.txt

git commit -m "Initial commit"



#ایجاد تغییرات uncommitted

echo "Change 1" >> file.txt
```


دانشکار

```
echo "New file" > new.txt
```

#کردن Stash

```
git stash

git status
```

#یا با پیام

```
git stash push -m "WIP: working on feature"
```

#ایجاد تغییرات دیگر

```
echo "Change 2" >> file.txt

git stash push -m "Another WIP"
```

#لیست stash ها

```
git stash list
```

#مشاهده جزئیات

دانشکار

```
git stash show

git stash show -p

git stash show stash@{1}
```

#Apply کردن stash

```
git stash apply

git stash apply stash@{1}
```

#یا pop (apply + delete)

```
git stash pop
```

#حذف stash

```
git stash drop stash@{0}
```

git stash clear حذف همه

فایل 05_stash/stash_explanation.md با توضیح:

- چرا بعضی تغییرات داخل stash نمی‌رود(untracked files)
- تفاوت apply و pop

دانشکار

- استفاده از git stash -u برایuntracked files

تحویل:

- خروجی git stash list وgit stash show

- توضیح در stash_explanation.mdدرباره رفتار stash باuntracked files

---

بخش ششم Diff :و — History چهار نمره

سوال ۶.۱ — git diff — دو نمره

آنچه می‌خواهم :استفاده از git diff برای مشاهده تغییرات

سناریو:

```
cd 06_diff_history

mkdir diff_demo && cd diff_demo

git init


# Setup

echo "Line 1" > file1.txt

echo "Line 1" > file2.txt
```


دانشکار

```
git add file1.txt file2.txt

git commit -m "Initial commit"



#تغییرات

echo "Line 2" >> file1.txt

echo "Line 2" >> file2.txt

git add file1.txt  # فقط file1 staged



#انواع diff

#        git diff تغییرات unstaged

#        git diff --staged تغییرات staged

#        git diff HEAD همه تغییرات نسبت به آخرین commit



#فیلتر بر اساس فایل

git diff file1.txt

git diff --staged file2.txt
```

#مقایسه *branches*

```
git branch feature

git switch feature

echo "Feature line" >> file1.txt

git add file1.txt

git commit -m "Feature change"


git switch main

git diff main feature

git diff main feature -- file1.txt
```

#مقایسه *commits*

```
git log --oneline

git diff HEAD~1 HEAD

git diff <commit-hash-1> <commit-hash-2>
```

تحویل:

- خروجی انواع git diff

دانشکار

- توضیح تفاوت git diff, git diff --staged, git diff HEAD در diff_explanation.md

سوال ۶.۲ — سفر در زمان — دو نمره

آنچه می‌خواهم :درک detached HEAD و کار با تاریخچه

سناریو:

```
cd 06_diff_history

mkdir time_travel && cd time_travel

git init


#ایجاد چند commit

echo "Commit 1" > file.txt

git add file.txt

git commit -m "Commit 1"


echo "Commit 2" >> file.txt

git add file.txt

git commit -m "Commit 2"
```

```
echo "Commit 3" >> file.txt

git add file.txt

git commit -m "Commit 3"
```

#مشاهده تاریخچه

```
git log --oneline
```

```
# Detached HEAD

git checkout HEAD~1

git status

git log --oneline
```

#ایجاد branch از detached HEAD

```
git branch from-detached
```

#بازگشت به main

```
git switch main
```

#استفاده از موقعیت HEAD

```
git checkout HEAD~2

git checkout HEAD^

git checkout HEAD~1
```

فایل 06_diff_history/detached_head.md با توضیح:

- مفهوم detached HEAD
- چرا detached HEAD اتفاق می‌افتد
- چگونه از detached HEAD خارج شویم

تحویل:

- خروجی git log و git status در حالت detached HEAD
- توضیح در detached_head.md

بخش هفتم: Reset و Revert — سه نمره

سوال ۷.۱ git restore و git reset — یک نمره

آنچه می‌خواهم: بازگرداندن تغییرات و پاک کردن commits

سناریو:

```
cd 07_reset_revert

mkdir reset_demo && cd reset_demo

git init


# Setup

echo "Version 1" > file.txt

git add file.txt

git commit -m "Commit 1"


echo "Version 2" > file.txt

git add file.txt

git commit -m "Commit 2"


echo "Version 3" > file.txt

git add file.txt

git commit -m "Commit 3"
```

```
git log --oneline


# Restore (بازگرداندن فایل از staging)

echo "Modified" > file.txt

git add file.txt

git restore --staged file.txt


# Restore (بازگرداندن از working directory)

git restore file.txt


# Reset (soft - فقط HEAD را تغییر می‌دهد)

git reset --soft HEAD~1

git status

git log --oneline


# Reset (mixed - HEAD و staging)

git reset HEAD~1
```

```
git status
```

```
# Reset (hard - همه چیز)
git reset --hard HEAD~1
git log --oneline
```

تحویل:

- خروجی git status وgit log بعد از هرreset
- توضیح تفاوت --soft, --mixed, --hard در reset_explanation.md

سوال ۷.۲ — git revert — دو نمره

آنچه می‌خواهم :برگرداندن تغییرات باrevert

سناریو:

```
cd 07_reset_revert
mkdir revert_demo && cd revert_demo
git init

# Setup
echo "Line 1" > file.txt
```

```
git add file.txt

git commit -m "Add line 1"


echo "Line 2" >> file.txt

git add file.txt

git commit -m "Add line 2"


echo "Line 3" >> file.txt

git add file.txt

git commit -m "Add line 3"


git log --oneline
```

Revert #آخرین commit

```
git revert HEAD

git log --oneline
```

Revert #یک commit خاص

دانشکار

```
git revert HEAD~1

git log --oneline


#نمایش تفاوت

git show HEAD
```

فایل 07_reset_revert/revert_vs_reset.md با توضیح:

- تفاوت git reset وgit revert
- چه زمانی از کدام استفاده کنیم
- چرا revert برای shared repositories بهتر است

تحویل:

- خروجی git logقبل و بعد ازrevert
- توضیح در revert_vs_reset.md

بخش هشتم — GitHub :پنج نمره

سوال ۸.۱ — اتصال به GitHub — دو نمره

آنچه می‌خواهم :ساخت repository در GitHub و push کردن

دانشکار

فایل 08_github/github_setup.md با توضیح:

- ساخت اکانتGitHub
- ایجاد(repository (public/private
- تنظیمremote
- Authentication (HTTPS با Personal Access Token یا SSH)

سناریو:

```
cd 08_github

mkdir github_demo && cd github_demo

git init


#ایجاد فایل‌ها

echo "# My Project" > README.md

echo "console.log('Hello');" > app.js

git add .

git commit -m "Initial commit"
```

#اضافه کردن remote

```
git remote add origin
https://github.com/YOUR_USERNAME/YOUR_REPO.git
```

#یا با SSH

```
git remote add origin
git@github.com:YOUR_USERNAME/YOUR_REPO.git
```

#بررسی remote

```
git remote -v
```

```
# Push
```

```
git branch -M main
```

```
git push -u origin main
```

#بررسی tracking

```
git branch -vv
```

تحویل:

دانشکار

- خروجی git remote -v وgit branch -vv
- توضیح upstream و branch tracking در github_setup.md

سوال ۸.۲ Pull — و — Collaboration دو نمره

آنچه می‌خواهم :کار با pull و حلconflict

سناریو:

cd 08_github

mkdir collaboration && cd collaboration

git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git

cd YOUR_REPO

#ایجاد تغییرات محلی

echo "Local change" > local.txt

git add local.txt

git commit -m "Add local file"

Pull (fetch + merge)

git pull origin main

#اگر *conflict* بود

#حل *conflict* و*commit*

```
git add .

git commit -m "Merge remote changes"
```

```
# Push

git push origin main
```

#تغییر*default branch behavior*

```
git config pull.rebase false  # merge (default)

git config pull.rebase true    # rebase

git config pull.ff only        # fast-forward only
```

فایل 08_github/collaboration.md با توضیح:

- اضافه کردن collaborator در GitHub
- تفاوت git pull و git fetch
- تنظیمات pull behavior

دانشکار

- نحوه نوشتن README بهتر

تحویل:

- خروجی git pull وgit push

- یک README.md نمونه با ساختار خوب در collaboration/README.md

---

تحویل نهایی — یک نمره

فایل :README.md

- Git Homework #هفته دهم

##ساختار پروژه

- 01_*basics: نصب، معرفی، اولیه repository*

- *02*_commits: git add, commit, amend

- 03_*branching: ایجاد و مدیریت branches*

- *04*_merging: merge ،conflict resolution

- 05_*stash: ذخیره موقت تغییرات*

- *06*_diff_*history: مشاهده تغییرات و تاریخچه*

دانشکار

- *07_reset_revert: بازگرداندن تغییرات*

- *08_github: کار با GitHub و collaboration*

##مفاهیم کلیدی

- Atomic commits

- Branching strategy

- Merge vs Rebase

- Conflict resolution

- GitHub workflow

##دستورات پرکاربرد

git init, git add, git commit

git branch, git checkout, git switch

git merge, git stash

git diff, git log

git reset, git revert

git remote, git push, git pull

نکات مهم:

- تمام دستورات git را در git_commands_log.txtذخیره کنید
- برای هر بخش توضیحات مفهومی در فایل‌های markdown بنویسید
- خروجی دستورات مهم را در فایل‌های txt ذخیره کنید
- README.mdرا کامل و حرفه‌ای بنویسید

سوالات متداول

تفاوت git checkout و git switch چیست؟ git switchبرای تغییر branch است، git checkoutبرای branch و restore فایل‌ها Git 2.23+ .توصیه می‌کند از switchو restoreاستفاده کنید.

چه زمانی از reset و چه زمانی از revert استفاده کنیم؟ resetبرای تغییرات local که push نشده‌اند، revertبرای تغییرات push شده در shared repository.

چرا بعضی فایل‌ها در stash نمی‌روند؟ فایل‌های untracked به صورت پیش‌فرض stash نمی‌شوند. از git stash -uاستفاده کنید.

detached HEADچیست؟ وقتی HEAD به یک commit خاص اشاره می‌کند نه به یک branch.برای خروج، یک branch جدید بسازید یا به branch موجود برگردید.

دانشکار

موفق باشید 🚀 !